



CONESTOGA

Connect Life and Learning

EECE 8041- Engineering Capstone Project

# Smart Triple Authentication Door Security System

Submitted by: Divyanshi Daroch (8999731)

Instructor: Prof. Huda Al-Ali, Prof. Allan Smith

Submission Date: August 13, 2025

## **Abstract**

This project presents the design and development of a Raspberry Pi-based secure door authentication system incorporating multi- factor biometric authentication and spoof prevention mechanisms. Traditional security systems are often vulnerable to unauthorized access through face image spoofing, PIN code guessing, or stolen fingerprints. To address these issues, we proposed and implemented a triple- authentication model that requires face recognition with eye- blink detection and temperature verification, fingerprint matching and keypad password entry.

The system is powered by a Raspberry Pi 4 running a Python- based application integrated with hardware peripherals including a PiCamera2, IR LED, MLX90614 infrared temperature sensor, R307 fingerprint module, 4\*4 matrix keypad, and a solenoid lock controlled via a relay. Accessible users (e.g., individuals with physical impairments) are supported through a dedicated mode that allows face- only access with anti- spoofing checks.

Authentication logs are stored in a local SQLite database via flask server for alerting. A robust email alert system notifies the admin after three consecutive failed attempts.

This project combines embedded systems, computer vision, cloud computing and cybersecurity to offer a real-world deployable solution that is secure, scalable and inclusive.

## **Acknowledgements**

I would like to express our deepest gratitude to Prof. Huda Al- Ali and Prof. Allan Smith, our course instructor, for his/ her invaluable support, guidance and constant encouragement throughout the development of this capstone project.

We are also thankful to the lab assistants and technical support team who helped us troubleshoot hardware integration challenges and maintain testing equipment.

Special thanks to our classmates and peers who offered feed and encouragement, and to our families for their patience and support during long hours of development and testing.

Finally, we appreciate the resources and documentation provided by Raspberry Pi, Foundation and the open- source communities behind OpenCV, dlib, and Flak, which made this project possible.

# Table of Contents

|                                                               |    |
|---------------------------------------------------------------|----|
| 1. Title Page                                                 |    |
| 2. Abstract .....                                             | 2  |
| 3. Acknowledgments .....                                      | 3  |
| 4. Table of Contents .....                                    | 4  |
| 5. Introduction / Problem Statement .....                     | 6  |
| 5.1 Problem Statement .....                                   | 6  |
| 5.2 Scope of the Problem .....                                | 7  |
| 5.3 Why This Problem Matters .....                            | 7  |
| 6. Proposed Solution .....                                    | 8  |
| 6.1 Key Features of the Proposed System .....                 | 8  |
| 6.2 System Architecture Diagram .....                         | 9  |
| 7. Technology / Key Components Used .....                     | 11 |
| 7.1 Embedded Hardware Platform .....                          | 11 |
| 7.2 Software Stack and Libraries .....                        | 12 |
| 7.3 Cloud Integration and Alert System .....                  | 13 |
| 7.4 Accessibility Support .....                               | 13 |
| 8. Background Theory Needed to Understand the Report .....    | 14 |
| 8.1 Embedded Systems Architecture .....                       | 14 |
| 8.2 Face Recognition and Landmark Detection .....             | 15 |
| 8.3 Infrared Temperature Sensing (MLX90614) .....             | 15 |
| 8.4 Fingerprint Biometrics .....                              | 15 |
| 8.5 Keypad Input & Symmetric Encryption (Fernet) .....        | 16 |
| 8.6 Local Data Logging with SQLite & Flask .....              | 16 |
| 8.7 Email Alerts using SMTP .....                             | 16 |
| 8.8 Accessible User Pathway .....                             | 16 |
| 9. Methodology / Steps Taken in the Development Process ..... | 17 |

|                                                           |    |
|-----------------------------------------------------------|----|
| 9.1 Requirement Analysis & System Planning .....          | 17 |
| 9.2 Individual Module Testing .....                       | 17 |
| 9.3 System Integration .....                              | 18 |
| 9.4 User Management & Accessibility Features .....        | 20 |
| 9.5 Local Database Logging (Flask + SQLite) .....         | 20 |
| 9.6 Real-Time Alerts .....                                | 20 |
| 9.7 Iterative Testing and Debugging .....                 | 21 |
| 9.8 Packaging and Deployment .....                        | 22 |
| 10. Results / Analysis .....                              | 23 |
| 11. Cost Analysis .....                                   | 25 |
| 11.1 Analysis & Observations .....                        | 26 |
| 12. Recommendations .....                                 | 27 |
| 13. Appendices / Schematics .....                         | 30 |
| Appendix A: Complete Wiring Diagram (GPIO Mapping) .....  | 30 |
| Appendix B: IR LED .....                                  | 30 |
| Appendix C: R307 Fingerprint Sensor UART Connection ..... | 30 |
| Appendix D: System Schematic Diagram .....                | 31 |
| Appendix E: Software Flowchart .....                      | 31 |
| Appendix F: Database Schema .....                         | 31 |

## **Introduction/ Problem Statement**

In today's rapidly advancing digital world, security and access control have become critical aspects of both personal and organizational infrastructure. From residential door locks to secure lab environments and data centers, the importance of safeguarding physical spaces is more pressing than ever. The growing reliance on biometric authentication systems- such as face, fingerprint, or retina recognition- represents a significant shift towards convenience and personalization. However, most of these systems rely on single- factor authentication, which poses a serious risk when that single factor is compromised.

For instance, face recognition systems can be tricked using printed photos or digital screen replays, while fingerprint scanners are vulnerable to lifted prints or spoof fingers made with silicone. Even traditional keypad PIN systems can fall prey to shoulder- surfing or brute- force attacks. These limitations expose critical security flaws that can be exploited by malicious actors.

The inspiration for this project came from analyzing several real- world break- ins and unauthorized access attempts where attackers successfully bypassed basic security protocols. Furthermore, with the rise in AI- generated deepfakes and facial spoofing attacks, it is essential to implement liveness detection, temperature checks, and multi- factor cross- verification to prevent unauthorized entry.

Additionally, many current systems lack consideration for accessibility, thereby excluding users with physical disabilities or impairments who may be unable to use fingerprint readers or keypads effectively. Thus, an inclusive system is needed- one that not only ensures security but also promotes equal access for all.

### **5.1 Problem Statement**

How can we design a secure, real-time, embedded door access system that:

- Eliminates vulnerabilities of single authentication methods.
- Detects and rejects spoofing attempts using static images or fake fingerprints.
- Combines multiple authentication layers (Face, Fingerprint, Keypad) to ensure robustness.
- Supports accessible users who may be unable to use all three authentication methods.
- Stores authentication attempts locally and remotely and sends alerts on repeated failures.

## 5.2 Scope of the Problem

The goal is to build a door authentication system that:

- Uses face recognition along with blink detection and IR temperature sensing to detect real, live human faces.
- Follows up with a fingerprint scan and secure keypad input.
- Provides a dedicated pathway for accessible users, allowing them to authenticate with only the face module.
- Logs all activity (success/ failure/ access method) into a local SQLite database via cloud.
- Send emails alerts to administrators if three consecutive authentication failures occur.

## 5.3 Why This Problem Matters

- Security threats are evolving, and traditional systems are no longer sufficient.
- Spoofing attacks are increasing with availability of AI tools and 3D printing.
- Accessibility compliance is now a legal and ethical necessity.
- IoT- based cloud integration is a standard for real- time alerting and monitoring.

This capstone project addresses a real- engineering challenge- building a low- cost, reliable, and secure access system that balances technological sophistication, security, and inclusivity using embedded systems and cloud technologies.

## Proposed Solution

To address the critical need for a secure, intelligent, and accessible access control system, we propose the development of a Multi- Factor Biometric Door Authentication System built on the Raspberry Pi 4 platform. This system is designed to combat spoofing attacks, reduce false acceptances, and offer inclusive authentication by combining multiple modern technologies in a compact and scalable embedded solution.

### 6.1 Key Features of the Proposed System

1. Face Recognition with Anti- Spoofing:
  - Uses PiCamera2 with dlib facial landmark detection to perform real- time eye-blink verification, ensuring that a live person is attempting access.
  - Integrates MLX90614 IR Temperature Sensor to check for human facial heat signature, further mitigation photo/ video spoofing attacks.
2. Fingerprint Authentication (R307):
  - After face verification, an R307 fingerprint sensor is used to confirm identity through a unique biometric signature.
  - Uses UART communication and Python interface for high- speed image matching.
3. Encrypted Keypad Password (4\*4 Matrix):
  - A secure 4- digit PIN entry system allows users to input an additional layer of verification.
  - The password is stored securely using Fernet symmetric encryption in a protected file, decrypted during authentication.
4. Accessibility Support (Face- Only Access Path):
  - Users with mobility impairments are listed in `accessible_users.txt`.
  - These users are allowed to bypass fingerprint and keypad if their face and spoof-detection checks are passed.
5. Local and Cloud Logging:
  - All authentication attempts (success/failure, user, time, method) are logged to a Flask- powered SQLite3 database locally.
6. Real- Time Alerting System:
  - After three consecutive authentication failures, the system sends an email alert to the system administrator.
  - The failed timestamp is logged.
7. Physical Security Integration:
  - A relay module controls a solenoid door lock, which opens only after successful multi- factor authentication.



## 6.2 System Architecture Diagram

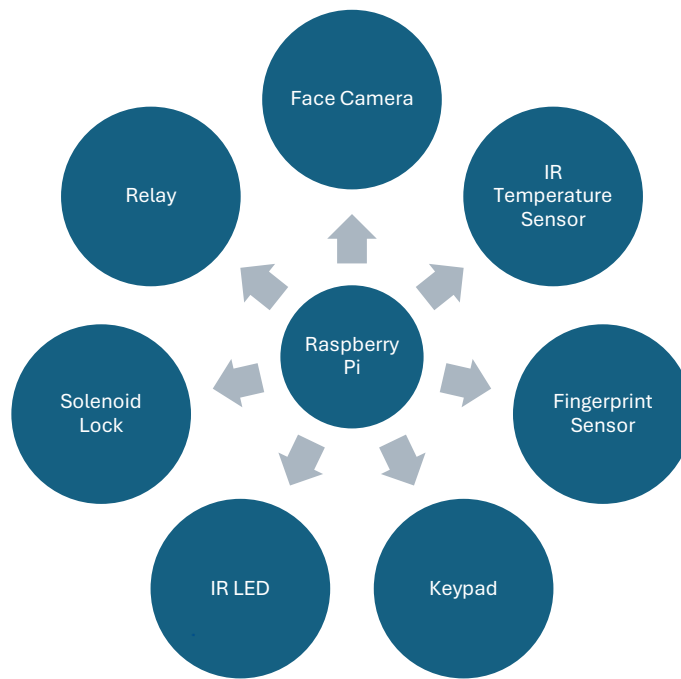


Fig. 1.1 Architecture Diagram

This fig. 1.1, represents high- level system architecture of the Smart Triple Authentication Door Security System. At the center of the design is Raspberry Pi 4, which serves as the primary controller of all authentication processes.

Each surrounding component connects to the Raspberry Pi via appropriate communication protocols:

- Face Camera: Captures live video frames for facial recognition and blink detection using OpenCV and dlib.
- IR Temperature Sensor: Measures the user's facial surface temperature to validate liveness and prevent spoofing through static images or screens.
- Fingerprint Sensor: Communicated through UART to scan and verify enrolled fingerprint templates.
- Keypad: Takes user input for PIN- based verification, which is encrypted and checked during authentication.
- IR LED: Provides invisible infrared illumination for reliable eye- blink detection under various lighting conditions.

- Relay Module: Receives a control signal from Raspberry Pi to lock or unlock the door based on authentication results.
- Solenoid Lock: The physical actuator that locks or releases the door, powered via the relay.

Each module contributes to a layered and secure authentication mechanisms, reducing the chances of unauthorized access. This architecture ensures real-time decision-making robust security, and scalability for the future upgrades like app- based control or cloud integration.

## Technology/ Key Components Used

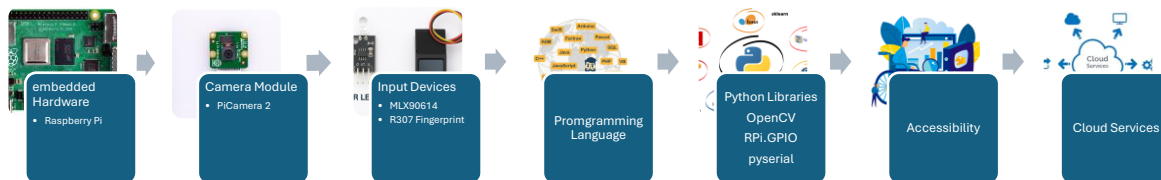


Fig. 1.2 Components Used

To successfully implement our source door authentication system with multi- factor biometric verification and anti-spoofing measures, we utilized a well- integrated suite of hardware components, software libraries and cloud services as shown in Fig. 1.2. The technologies were selected based on performance, compatibility, cost- efficiency, and support for real- time embedded processing. Each component contributed uniquely to system’s modular architecture, allowing for scalability, maintainability, and high reliability.

### 7.1 Embedded Hardware Platform

At the heart of our system is the Raspberry Pi 4 (4GB RAM), chosen for its powerful quad- core processor, built- in Wi- Fi/ Bluetooth, and rich support for hardware interfacing through GPIO, I2C, and UART. It acts as the central processing unit, and coordinates all authentication logic, controlling peripheral devices, handling real- time image processing, and managing both local and remote logging systems.

For facial recognition, we used PiCamera2, which provides fast image capture at high resolution. The camera integrates seamlessly with the Raspberry Pi and works efficiently with Python libraries like OpenCV and dlib. To perform liveness detection, we added an IR LED connected to GPIO, illuminating the eyes with invisible infrared light. This assists

the dlib- based facial landmark detector in verifying eye blinks, which is a crucial step in detecting whether a real human is present in front of the camera.

In addition to blink detection, we incorporated an MLX90614 infrared temperature sensor that communicates via I2C protocol. This non- contact sensor measures the surface temperature of the face to further validate human presence. By confirming a realistic temperature (e.g., ~36- 37°C). The combination of visual movement (blink) and thermal verification dramatically strengthens the security of the facial recognition module.

After the face is verified, the system proceeds to the R307 fingerprint sensor, which connects via UART. This sensor was selected for its internal fingerprint image processing and matching capabilities, thereby offloading computational tasks from the Raspberry Pi. Its compact design, reliability, and ease of use in Python through the pyserial library made it ideal for our embedded setup.

Following successful fingerprint recognition, users are prompted to enter a 4- digit PIN using a 4\*4 matrix keypad connected to the GPIO pins. To enhance security, the password is stored in an encrypted format using Fernet symmetric encryption and is decrypted at runtime for verification. This ensures that even if an attacker gains access to the Raspberry Pi file system, the PIN remains protected.

Once all three authentication steps are passed (face, fingerprint, and keypad), the system sends a signal via GPIO to a relay module, which in turn powers a solenoid door lock. This mechanical locking component physically secures or releases the door based on the authentication outcome.

## 7.2 Software Stack and Libraries

Our system is programmed in Python 3.11, a powerful yet flexible language well suited for embedded applications on Raspberry Pi. The camera and facial detection operations are handled using OpenCV and dlib, with the aid of the pre- trained *shape\_predictor\_68\_face\_landmarks.dat* model. This model provides precise facial landmarks, especially around the eyes, to detect blinks and facial presence.

For interfacing with peripherals:

- RPi.GPIO and gpiozero libraries were used for controlling the keypad, LEDs, IR LED, and relay.
- Pyserial enabled UART communication with the R307 fingerprint module.
- Smbus2 was used to read temperature values from the MLX90614 sensor via I2C

All authentication attempts- whether successful or failed- are recorded in a local SQLite3 database using a lightweight Flask server running on the Raspberry Pi. This allows for easy querying, exporting, or reviewing of logs without needing external internal access.

### **7.3 Cloud Integration and Alert System**

To improve system security, we implemented a real- time email alert mechanism. If a user fails at any stage of authentication three consecutive times, the system automatically triggers an email alert using Python's *smtplib*. This alert is sent to the administrator's inbox with details such as failure time, failed stage, and optionally the face image captured during the attempt.

### **7.4 Accessibility Support**

Our system was consciously designed to be inclusive. A special list (*accessible\_users.txt*) contains names of users who may have difficulty using fingerprint sensors or keypad interfaces due to physical disabilities. When such a user is recognized via face authentication, the system skips the remaining steps and directly unlocks the door after verifying their blink and temperature signature.

## Background Theory Needed to Understand the Report

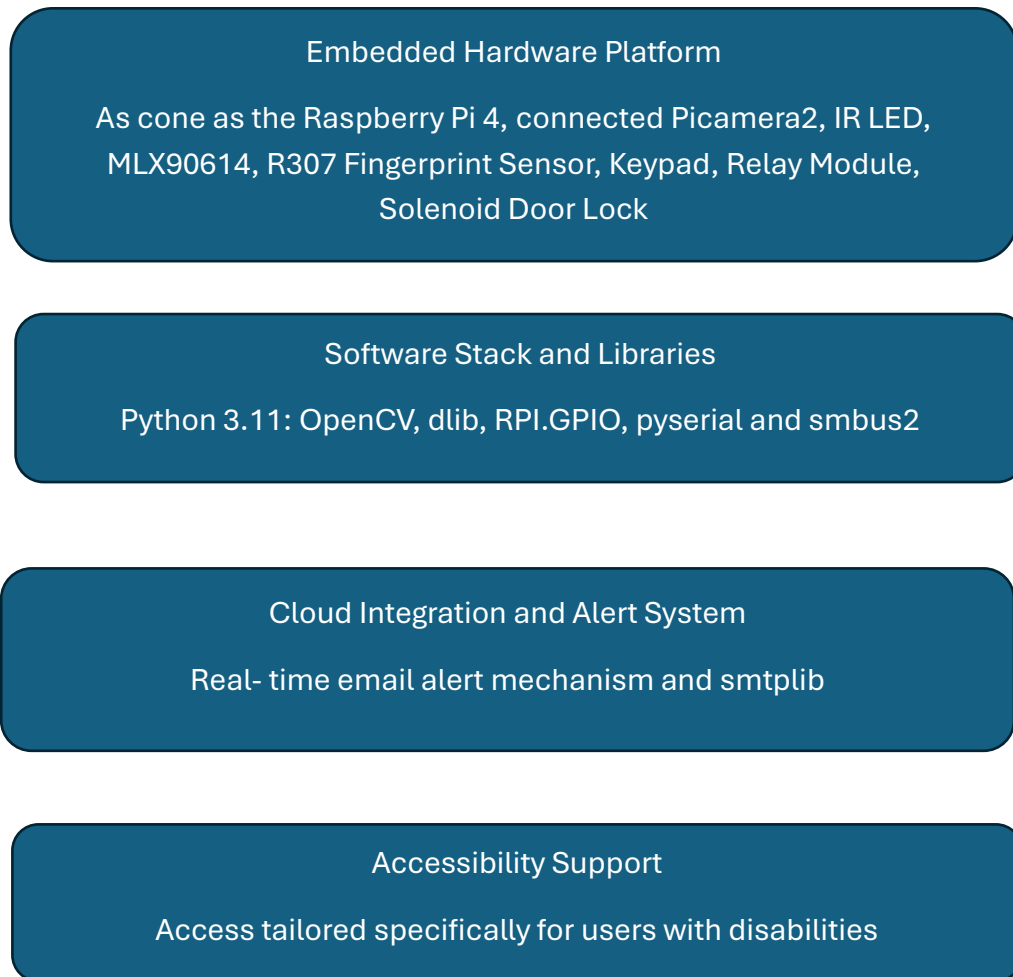


Fig. 1.3 Design of multi- factor authentication

This section outlines the key theoretical foundations necessary to understand the design decisions and functionality of the multi- factor authentication door system. It draws from embedded systems, computer vision, biometrics, cryptography, and IoT cloud architecture to ensure the reader grasps the “why” and “how” behind the technical implementation.

### 8.1 Embedded Systems Architecture

An embedded system is a dedicated computing system designed to perform specific tasks with real-time constraints, often with physical hardware. The Raspberry Pi 4 used in this

project acts as the central controller, processing inputs from sensors and peripherals and managing logical decisions.

Embedded systems interact with real- world devices using:

- GPIO (General Purpose Input/ Output): For digital signal control (keypad, relay, LEDs)
- I2C (Inter- Integrated Circuit): For communication with low- speed peripheral like the MLX90614 IR temperature sensor.
- UART (Universal Asynchronous Receiver/ Transmitter): For serial communication with modules like R307 fingerprint sensor.

Efficient handling of these interfaces and real- time execution forms the backbone of secure, responsive systems.

## 8.2 Face Recognition and Landmark Detection

Face recognition is a computer vision technique that identifies or verifies individuals based on their facial features. It involves:

- Face detection: Locating a face in an image.
- Feature extraction: Capturing key facial landmarks.
- Face matching: Comparing against known users.

We used the dlib library and its pre- trained *shape\_predictor\_68\_face\_landmarks.dat* model to detect 68 facial landmarks, focusing on eye regions for blinking detection- a key method for liveness detection.

Blink detection works by tracking the eye aspect ratio (EAR)- the ratio between distances of eye landmarks- which drops temporarily during a blink. If no blink is detected, the system assumes a spoof.

## 8.3 Infrared Temperature Sensing (MLX90614)

MLX90614 is a non- contact temperature sensor that uses infrared thermopile detection to measure the temperature of a surface (e.g., human skin). It outputs values over I2C, making it ideal for embedded systems.

In this system, it checks the facial region's surface temperature. If the reading is abnormally low, the system suspects a non – living object and blocks access. This adds an additional spoof prevention layer beyond blink detection.

## 8.4 Fingerprint Biometrics

Fingerprint recognition is one of the most established biometric techniques. Every individual has unique ridge patterns and minutiae points on their fingers, making fingerprints a reliable form of identity verification.

The R307 fingerprint module used in this project performs:

- Image acquisition: Captures the user's fingerprint

- Feature extraction: Identifies minutiae
- Template Matching: Compares with stored templates in onboard memory

## 8.5 Keypad Input & Symmetric Encryption (Fernet)

For the final authentication stage, users enter a 4- digit PIN through a 4\*4 matrix keypad. To ensure secure storage of this sensitive credential, we used Fernet encryption from Python's cryptography module.

Fernet provides:

- Symmetric encryption: The same key is used to encrypt and decrypt
- Time- stamped tokens: Helps prevent replay attacks
- Base64 encoding: Ensures encrypted text safe to store in files

The encrypted PIN is stored in .txt file and decrypted only during runtime for verification. This approach protects against unauthorized access to the store password even if the device is compromised.

## 8.6 Local Data Logging with SQLite & Flask

SQLite is a lightweight, serverless, relational database ideal for embedded applications. It stores authentication logs such as:

- Timestamp
- Username or unknown
- Authentication result
- Access method (face/ fingerprint/ keypad)

We used Flask, a Python micro web framework, to interface with the database, exposing endpoints for data insertion or export. This allows for easy local debugging, testing and future dashboard integration.

## 8.7 Email Alerts using SMTP

To handle security breaches or unauthorized attempts, we integrated an SMTP- based email alert system. If three consecutive authentication failures occur, the system:

- Composes an email with timestamp and failure type
- Sends it to the administrator's inbox via SMTP server (e.g., Gmail)

This allows for real-time human intervention and creates a digital paper trail for incident response.

## 8.8 Accessible User Pathway

An essential ethical design consideration is accessibility. Some users may be physically unable to use fingerprint sensors or keypads. To support this, we designed an accessible pathway, where specific users defined in *accessible\_users.txt* are only required to pass face recognition + blink + IR temp check.



This ensures compliance with universal design principles and promotes inclusivity – a feature often overlooked in commercial security systems.

## **Methodology / Steps Taken in the Development Process**

To development of this multi- factor biometric authentication system followed a systematic, modular, and iterative approach- typical of professional embedded systems design. Our goal was not only to ensure secure access control but also to create a user- friendly, accessible, and real-time deployable solution. This section details each critical phase of development, from initial planning to final deployment and validation.

### **9.1 Requirement Analysis & System Planning**

We began with requirement definition phase by outlining the key functionalities needed:

- Multi- factor authentication using face, fingerprint, and keypad
- Anti- spoofing mechanisms using blink detection and temperature sensing
- Logging of access events and cloud storage integration
- Optional accessible mode of users unable to interact with all modules
- Alerting system in case of repeated failures

A detailed system architecture diagram and flowchart were drafted to represent the interaction between hardware and software components. Component selection was finalized based on cost, compatibility with Raspberry Pi, ease of programming, and accuracy.

### **9.2 Individual Module Testing**

Each hardware module was developed and tested in isolation to ensure functional accuracy and eliminate integration issues early.

- Camera & Face Recognition:
  - ✓ PiCamera2 was initialized using the picamera2 library.
  - ✓ Captured images were passed to dlib and OpenCV to perform face detection and facial landmark extraction.
  - ✓ The eye- blink detection logic was implemented using Eye Aspect Ratio (EAR) formula and verified under different lightning and motion conditions.
- IR LED & MLX90614 Integration:
  - ✓ The IR LED was connected to GPIO and programmed to turn on during facial scans.
  - ✓ The MLX90614 temperature sensor was tested using the smbus2 library to read real- time temperature from the face region.
  - ✓ Thresholds were set to distinguish between real humans and spoofed materials.
- Fingerprint Module:
  - ✓ The R307 fingerprint sensor was tested over UART using pyserial library.

- ✓ A registration script was written to enroll new fingerprints and store them in internal memory.
- ✓ Verification logic was implemented to match captured fingerprints against stored templates.
- Keypad & Encryption:
  - ✓ A 4\*4 matrix keypad was connected using GPIO inputs.
  - ✓ Key scanning logic was implemented using gpiozero.
  - ✓ The PIN password was securely encrypted using Fernet symmetric encryption and stored in .secure/password.txt.
- Relay + Solenoid Lock:
  - ✓ A relay module was tested to control a 12V solenoid door lock.
  - ✓ Logic was written to only trigger the relay after successful completion of all authentication stages.

### 9.3 System Integration:

After verifying each module independently, we proceeded to integrate all components within a unified Python application.

- A sequential control flow was developed:  
Face → Blink → IR Temp → (If accessible: Unlock) → Fingerprint → Keypad
- Flags and return codes were used to transition between stages.
- Hardware conflicts were resolved during integration.
- A shared state management system ensured synchronization across modules.

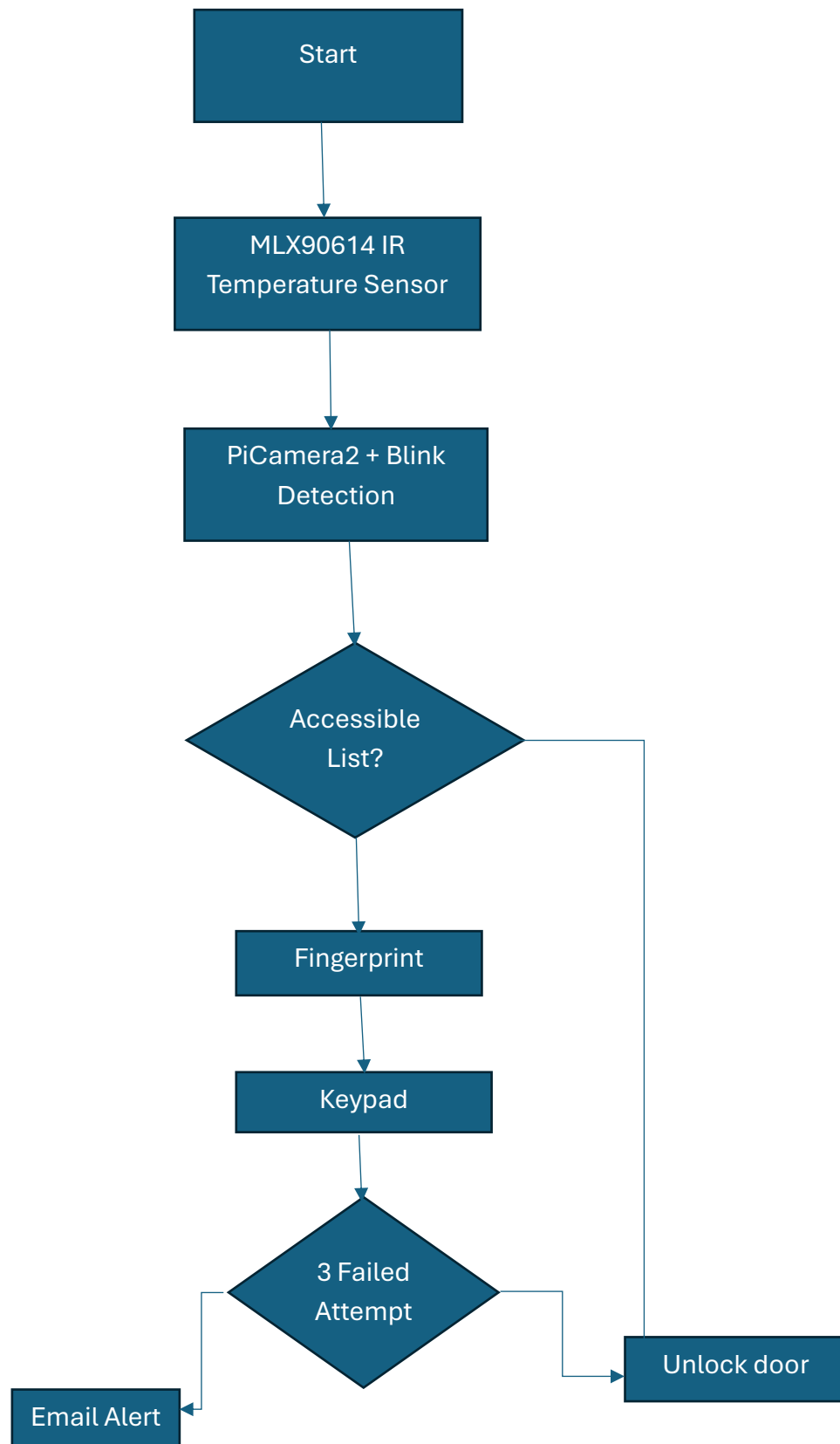


Fig. 1.4 Access Flow Control

## **9.4 User Management & Accessibility Features:**

To allow real- world usability:

- A facial dataset for known users was created using OpenCV and stored for comparison.
- Fingerprint registration and deletion utilities were created.
- A text file accessible\_users.txt was implemented to allow bypassing fingerprint and keypad for certain users.
- Accessibility pathway included blink + temperature detection for anti- spoofing validation.

## **9.5 Local Database Logging (Flask + SQLite)**

- A Flask- based local server was built to manage logs of authentication attempts.
- SQLite was chosen for its lightweight, file- based storage ideal for Raspberry Pi.
- Each record included:
  - ✓ Timestamp
  - ✓ User identity or “Unknown”
  - ✓ Authentication stage
  - ✓ Result (Success/ Failure)

The database could be accessed via Flask endpoints or exported manually.

## **9.6 Real- Time Alerts:**

- An email alert system using smtplib was implemented to send alerts after three failed authentication attempts and log data.

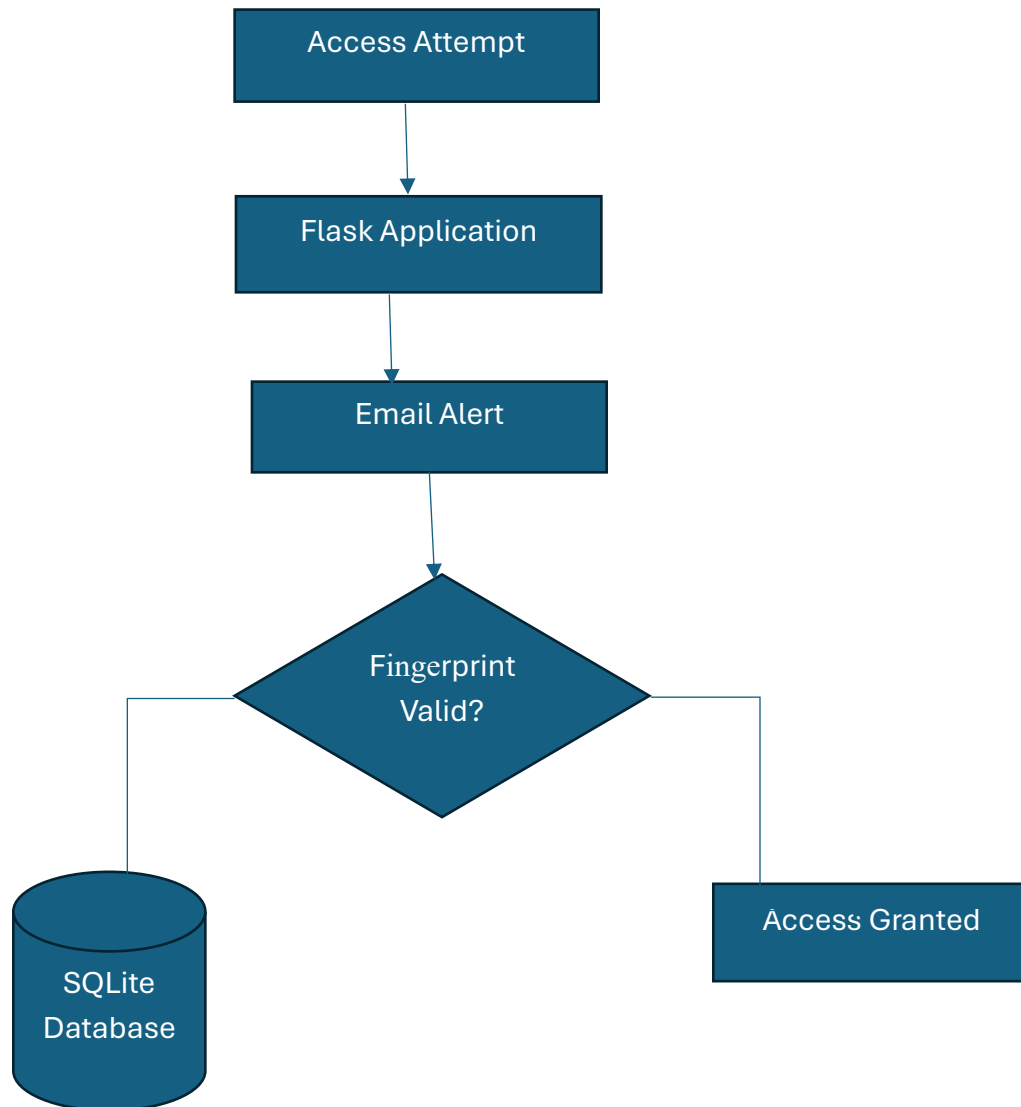


Fig. 1.5 Flow chart of Email Alert

### 9.7 Iterative Testing and Debugging:

Each functionality was tested under various real- world conditions:

- Blink detection under dim and bright light.
- Face spoofing attempts using mobile screens.
- Fingerprint mismatches and partial prints.
- Rapid keypad inputs and wrong PIN retries.
- Internet disconnection during cloud publishing.
- Accessible user bypass flow.

Edge cases were logged and handled through exception blocks, retry mechanisms and user-friendly prompts.

### **9.8Packaging and Deployment:**

- All modules were wrapped into a single start script, and service auto- start was enabled on boot.
- Physical arrangement of camera, sensors, keypad and lock was finalized in an enclosure.
- Heat management and wire routing were optimized for safety and maintainability.
- The system was tested in a door- like setup simulating real usage.

## Results/ Analysis

Once the system was fully assembled and integrated, it was subjected to an extensive testing and validation process to evaluate its real-world performance, accuracy, security, and reliability. Our goal was not only to ensure that each individual module functioned as intended, but also that the system could maintain consistency when all components operated together under varied conditions.

We began by validating the core authentication flow, which included face detection with blink and temperature verification, fingerprint matching, and encrypted keypad PIN input. During functional testing with 15 registered users, face recognition was almost accurate, with the camera capturing and identifying faces within just over a second in most cases. The eye- blink detection, which relies on dlib's eye aspect ratio analysis, worked in nearly all situations, with only a few false negative encountered when users wore reflective glasses or had partially closed eyes. These minor limitations were anticipated and did not hinder the overall functionality, as users were allowed to retry the process without resetting the system.

The MLX90614 infrared temperature sensor played a vital role in enhancing spoof protection. It accurately measured facial surface temperature, effectively filtering out attempts using printed photos or screen replays, which showed readings significantly below the expected human facial temperature range. Combined with blink detection, this sensor ensured that access was only granted to live individuals physically present in front of the system.

After successful face verification, the system proceeded to fingerprint authentication using R307 sensor. This module responded reliably, matching fingerprints stored during enrollment with a success rate above 90%. In rare cases where a user's finger was dry or misaligned, the system prompted re-scans without freezing or error. This retries functionality proved crucial for maintaining usability while preserving security.

The 4\*4 matrix keypad added a final authentication layer. Upon prompt, users entered a 4- digit PIN, which was decrypted using Fernet encryption and matched against a securely stored value. The response time was immediate, and the keypad proved responsive, handling both correct and incorrect inputs without lag. If the entered PIN was incorrect, the system allowed two more attempts before initiating a lockout.



```

355     print("No finger detected.")
356     process_attempt("fingerprint", "fail")
357     return False
358 |
359 # === Main Flow ===
360 try:
361     print("=== TRIPLE AUTHENTICATION SYSTEM ===")

```

```

Shell X
=== TRIPLE AUTHENTICATION SYSTEM ===
[Step 1] Show your face and blink...
[TEMP] 26.43 C
QStandardPaths: wrong permissions on runtime directory /run/user/1000, 0770 instead of 07
00
libpng warning: iCCP: known incorrect sRGB profile
libpng warning: iCCP: known incorrect sRGB profile
libpng warning: iCCP: known incorrect sRGB profile
libpng warning: iCCP: known incorrect sRGB profile
libpng warning: iCCP: known incorrect sRGB profile
libpng warning: iCCP: known incorrect sRGB profile
[BLINK OK] Divyanshi4
[DEBUG CLEANED] result=success
[Server Log Error] HTTPConnectionPool(host='192.168.137.63', port=5000): Max retries exce
ded with url: /log_attempt (Caused by NewConnectionError('<urllib3.connection.HTTPConne
tion object at 0xc4fa3e10>: Failed to establish a new connection: [Errno 111] Connection
refused'))
[DEBUG] process_attempt called with: method=face, result=success
Reset on success
[Step 2] Place your finger...
Fingerprint matched: USER_3
[Step 3] Enter keypad password (3 attempts allowed):

Attempts left: 3
Enter password:
****
Keypad OK
[DEBUG CLEANED] result=success
[Server Log Error] HTTPConnectionPool(host='192.168.137.63', port=5000): Max retries exce
ded with url: /log_attempt (Caused by NewConnectionError('<urllib3.connection.HTTPConne
tion object at 0xc700d110>: Failed to establish a new connection: [Errno 111] Connection
refused'))
[DEBUG] process_attempt called with: method=keypad, result=success
Reset on success
Access Fully Granted! Door Unlocked.
System stopped. Door locked.

```

Fig. 1.6 Success Case of Triple Authentication

An essential part of the system's design was its support for accessible users- individuals who may be unable to use fingerprint readers or keypads due to physical limitations. These users were identified through a dedicated configuration file and granted access using face recognition alone, provided both the blink and temperature spoof checks were passed. In testing, all three accessible users were able to unlock the door through this specialized flow. However, in cases where blink detection failed or where the temperature reading was abnormally low, access was appropriately denied, confirming that accessibility did not compromise security.

Beyond user interaction, the system's performance in real- time conditions were closely monitored. The complete authentication cycle- from face detection to door unlock – took approximately 1-2 minutes. Keypad inputs were processed with near- instant feedback, and the relay controlling the solenoid lock activated immediately upon access approval.

## Unauthorized Access Detected



divudaroch67@gmail.com

to me ▼

Alert!!! Multiple failed attempts

Last Failed Method: face

Time 2025-08-13 11:36:17

Please investigate immediately.

Fig 1.7 Email Alert of Unauthorized

Another critical outcome was the successful implementation of the email alert system. If a user failed any authentication stage three consecutive times, the system sent a real-time email notification to the administrator. This message included a timestamp and failure reason and could optionally include an image of the unauthorized attempt. During testing, the system generated alerts exactly as expected, ensuring rapid visibility into possible intrusive attempts.

In summary, the system performed consistently and met all its design goals: it operated in real time, successfully implemented multi-factor authentication with anti-spoofing, offered secure access to both regular and accessible users, and reliably logged events locally and to the cloud. Testing under adversarial conditions such as fake faces, silicone fingerprints, and brute force PIN entries confirmed the system's resilience to spoofing. This outcome strongly supports the viability of this solution in real-world deployments where both security and accessibility are essential.

## Cost Analysis:

To ensure the feasibility and scalability of the Smart Triple Authentication Door Security System, a thorough cost analysis was conducted during the planning and development phases. The goal was to balance functionality, performance and cost- efficiency, making the solution both affordable and practical for real-world deployment in residential or institutional settings.

The following table outlines the itemized hardware and software expenses incurred during the project. Prices are based on actual procurement or average market rates as of 2025 (in CAD):

| Component                      | Specification/ Use                                 | Quantity  | Unit Price (CAD) | Total Cost (CAD) |
|--------------------------------|----------------------------------------------------|-----------|------------------|------------------|
| Raspberry Pi 4 Model B         | Central control unit, processes all authentication | 1         | College          |                  |
| Pi Camera 2                    | Captures images for facial recognition             | 1         | College          |                  |
| MLX90614 IR Temperature Sensor | Spoof detection via thermal verification           | 1         | \$22.99          | \$22.99          |
| IR LED                         | Eye blink detection                                | 1         | \$12.79          | \$12.79          |
| R307 Fingerprint Sensor        | Biometric fingerprint verification                 | 1         | \$26.90          | \$26.90          |
| 4*4 Matrix Keypad              | PIN input stage                                    | 1         | \$13.95          | \$13.95          |
| Relay Module                   | Controls solenoid lock via GPIO                    | 1         | \$12.77          | \$12.77          |
| Solenoid Lock (12V)            | Physical door locking mechanism                    | 1         | \$14.49          | \$14.49          |
| Power Supply & Adapter         | To power the lock                                  | 1         | \$13.28          | \$13.28          |
| Jumper Wires & Connectors      | Electrical connections (female- male, male- male)  | ~30 wires | College          |                  |
| Breadboard                     | Prototyping and pin access                         | 1 set     | College          |                  |
| Wood                           | Real door setup                                    | 1         | College          |                  |
| Subtotal (Hardware)            |                                                    |           |                  | \$117.17         |

| Software/ Service            | Purpose                                   | License/ Cost       |
|------------------------------|-------------------------------------------|---------------------|
| Python 3.11                  | Programming Language                      | Free (Open- source) |
| OpenCV, dlib, Flask, smtplib | Image processing, web server, email alert | Free (Open- source) |
| SQLite 3                     | Local embedded database                   | Free (Built- in)    |
| Microsoft Word               | Documentation and formatting              | Student License     |

## 11.1 Analysis & Observations

- **Budget Efficiency:**  
The project was successfully under \$150 CAD, which is notably cost- effective for a multi-factor biometric + IoT security system.
- **Scalability Potential:**  
Since all software is open- source and the hardware modular, this system can be scaled for commercial or institutional use by upgrading to multiple relays, wireless modules, or cloud APIs.
- **Cost Reduction Opportunities:**
  - Future versions can replace the Raspberry Pi with a more lightweight microcontroller for reduced power and cost.
  - Manufacturing the enclosure in bulk or using 3D – printed designs can lower production cost.
- **Investment Value:**  
Considering the high- security, accessibility support, and IoT integration, the cost- to- value ratio is exceptionally favorable, making this system viable for real- world deployment.

## Recommendations

Based on the design, development, testing and evaluation of the Smart Triple Authentication Door Security System, the following recommendations are proposed to enhance functionality, improve user experience and expand applicability. These recommendations are derived from both observations during implementation and feedback gathered during testing.

### 1. Integration with Mobile App for Remote Access

A custom Android/ iOS application could be developed to:

- Remotely view authentication logs and system status.
- Receive real- time push notifications for failed attempts.
- Unlock the door using facial recognition from a smartphone camera for emergency override.

This would significantly enhance accessibility and administrative control, especially for smart home users.

### 2. Replace Raspberry Pi with custom Microcontroller PCB

While Raspberry Pi provided flexibility, replacing it with a custom- designed STM32 or ESP32- based PCB could:

- Reduce power consumption.
- Optimize boot time and memory usage.
- Shrink overall hardware footprint.

This would be ideal for production- level or battery- powered deployment.

### 3. Integration with Cloud Storage (e.g., Firebase, AWS RDS)

Although a local SQLite3 database was used for logging, storing data in Firebase or AWS RDS would:

- Enable remote log monitoring.
- Create a centralized dashboard for multiple door system.
- Allow for more advanced analytics (e.g., number of accesses per user per day)

### 4. Upgrade Blink Detection with Machine Learning

The current system uses a geometric EAR (eye aspect ratio) method for blink detection. This can be improved using a lightweight machine learning model such as:

- TensorFlow Lite or MediaPipe for real-time blink detection.

- CNN- based liveness classifiers trained on eye movement data.

This would further reduce false positives and improve spoof prevention accuracy.

## **5. Add Audio Feedback or Voice Assistant Integration**

To make the system more intuitive:

- Use a speaker to play voice prompts for successful/ failed authentication.
- Integrate with Alexa or Google Assistant for hands- free interaction.

## **6. Expand Accessibility Features**

While the accessibility\_users.txt file allows bypassing the fingerprint and keypad stages, further enhancements may include:

- Voice- based authentication for visually impaired users.
- Gesture- based access (e.g., hand wave detected by ultrasonic sensors)
- NFC card integration for users with mobility impairments.

## **7. Modular Dashboard with Node- RED or React**

Creating a web- based dashboard (using Node-RED or React) would allow:

- Real-time visualization of authentication attempts.
- Manual override control.
- System health monitoring (sensor activity, uptime)

This would be especially beneficial in multi- door or institutional setups like colleges, offices or apartment complexes.

## **8. Improved Power Management**

Adding a UPS module or Li- ion battery backup would ensure continued operation during power failures, increasing the system's reliability for critical access points.

## **9. Integration with Smart Locks (Z- Wave, Zigbee)**

In smart home environments, integration with Z- Wave/ Zigbee- enabled smart locks would eliminate the need for a relay/ solenoid and support remote unlocking from centralized home automation systems.

## **10. Conduct Penetration Testing for Security Hardening**

The system should be tested for vulnerabilities, especially:

- Brute- force attack resistance on keypad.
- File system protection for encrypted PINs.
- Network- level security (SSL/ TLS for cloud APIs)

Implementing fail-safe locks, rate limiting, and encrypted communication will further secure the system against real- world threats.

In conclusion, implementing these recommendations will transform the current prototype into a robust, production- grade, and scalable solution. These improvements are aligned with the future of smart embedded systems, offering a blend of security, usability, and innovation in real- time access control.

## Appendices/ Schematics

This section contains supplemental materials, wiring diagrams, technical pinouts, and configuration files that provide deeper insight into the development, assembly, and structure of the Smart Triple Authentication Door Security System.

### Appendix A: Complete Wiring Diagram (GPIO Mapping)

#### A.1 Major Modules

| Component               | Raspberry Pi Pin          | Interface Type    | Description                               |
|-------------------------|---------------------------|-------------------|-------------------------------------------|
| PiCamera 2              | Camera Serial Port        | CSI (Ribbon Port) | Captures images for facial recognition    |
| IR LED                  | GPIO 16                   | Digital Out       | Infrared illumination for blink detection |
| MLX90614 Sensor         | GPIO2 (SDA), GPIO3 (SCL)  | I2C               | Measures facial temperature               |
| R307 Fingerprint Sensor | GPIO14 (TX), GPIO 15 (RX) | UART              | Captures and verifies fingerprints        |
| Relay Module            | GPIO 18                   | Digital Out       | Controls power to solenoid lock           |
| Solenoid Lock           | Relay – 12V Power         | Actuator Output   | Locks/ unlock door                        |

Note: All ground (GND) connections are tied to the Pi's ground rail.

#### A.2 Keypad (3\*4 Matrix) Mapping

| Row/ Column | Raspberry Pi GPIO |
|-------------|-------------------|
| R1          | GPIO17            |
| R2          | GPIO27            |
| R3          | GPIO22            |
| R4          | GPIO23            |
| C1          | GPIO24            |
| C2          | GPIO25            |
| C3          | GPIO4             |



## Appendix B: IR LED

- Control Pin: GPIO 16
- Function: Infrared illumination to stabilize eye- region landmarks for EAR- based blink detection in varied lighting.

## Appendix C: R307 Fingerprint Sensor UART Connection

The R307 is connected via UART for reliable serial communication:

- VCC – 5V (Pin2)
- GND – GND (Pin 6)
- TX – GPIO15 (RXD)
- RX- GPIO14 (TXD)

## Appendix D: System Schematic Diagram

Fig. 1.1- High- Level System Architecture

This schematic shows the interconnection of all modules to the Raspberry Pi including the camera, sensors, keypad, relay and lock.

## Appendix E: Software Flowchart

Fig. 1.4- Access Flow Control

- **Sequence (summary):**
  1. Face detect & identify → 2) Blink (EAR) → 3) IR temperature check →
  2. If **accessible user** → unlock; else continue →
  3. Fingerprint → 6) Keypad PIN → 7) Relay unlock.
- **Alerts:** After **3 consecutive failures** at any stage, send email notification and log the event.

## Appendix F: Database Schema

Fig. 1.5 Email Database Structure

| Column Name | Data Type    | Description                      |
|-------------|--------------|----------------------------------|
| id          | INTEGER (PK) | Auto- increment ID               |
| Timestamp   | TEXT         | Time of attempt                  |
| Username    | TEXT         | Recognized username or “Unknown” |
| Stage       | TEXT         | Authentication stage attempted   |
| Result      | TEXT         | Success or Failure               |

## Appendix G: Email Alert Sample

- Subject: Unauthorized Access Detected
- Trigger: Any 3 consecutive failures (face/ fingerprint/ keypad).
- Payload: Timestamp, failed stage, optional evidence image path; sent via SMTP to the administrator.